



INSTITUTO FED. DE EDUCAÇÃO, CIÊNC. E TEC. DE PERNAMBUCO
CURSO: TEC. EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DISCIPLINA: ALGORITMOS E ESTRUTURAS DE DADOS
PROFESSOR: RAMIDE DANTAS
ASSUNTO: PILHAS E FILAS.

Prática 03

Parte 1: Implementando Pilhas e Filas

Problema 1 (arquivo `pilha_min.cpp`): Problema MinStack ([LC155](#))

Implemente uma pilha especial que retorna o menor elemento presente na pilha até aquele momento.

Orientações:

- Use `vector<>` com a lógica usada em sala (não use `push_back`, etc.).
- Faça com que não seja necessário percorrer toda a pilha para encontrar o mínimo. (dica: registrar o mínimo até o momento para todos os elementos)

Desafio (Opcional): Adapte e submeta sua solução no LeetCode.

Problema 2 (arquivo `fila_media.cpp`): Fila com Média

Implemente uma fila especial que retorna a média atual dos elementos presentes nela.

Orientações:

- Implemente a fila de forma eficiente com *buffer* circular (ver slides).
- Faça com que não seja necessário percorrer toda a fila para recalcular a média.

Parte 2: Usando Pilhas e Filas

Problema 1 (arquivo `parenthesis.cpp`): Validação de Parênteses ([LC 20](#)).

Dada uma string contendo caracteres de abertura e fechamento de parênteses ‘(’ & ‘)’, colchetes ‘[’ & ‘]’ e chaves ‘{’ & ‘}’, diga se a string é válida. Exemplos:

- “{[()] }” : válida
- “()[]{}” : válida
- “{[(] }” : inválida
- “([)]{}” : inválida

O procedimento deve usar uma pilha (pode ser `stack<char>` ou `vector<char>`) para verificar a validade. Ao encontrar um caractere de abertura ((, [ou {), ele deve ser empilhado. Ao encontrar um caractere de fechamento () ,] ou }) o topo da pilha é desempilhado e deve corresponder ao caractere de fechamento, do contrário a string é inválida (se a pilha estiver vazia, também será inválida). Ao final, a pilha deve estar vazia para a string ser válida.

OBS.: Desconsidere a precedência entre parênteses, colchetes e chaves. Use o código abaixo para testar.

Desafio 1 (Opcional): Adapte e submeta seu código no LeetCode.

Desafio 2 (Opcional): Resolva o problema Notação Polonesa Reversa ([LeetCode 0150](#)).

Problema 2 (arquivo round_robin.cpp): Simulando o Round Robin

Implemente a função `round_robin()`, que simula a execução do algoritmo de escalonamento simplificado. Essa função recebe uma lista de processos com seus PIDs e demanda computacional (número de ciclos de CPU). Os processos devem ser colocados em uma fila (`queue<>`) e servidos em sequência pela CPU. Cada vez que um processo tem a CPU, é decrementado `max_ciclos` do número de ciclos do processo. Processos que zeram sua demanda de ciclos saem da fila e são registrados (PID) no `vector<>` de concluídos; os que ainda tem demanda, são colocados ao final da fila para a próxima rodada de processamento. A função `round_robin()` termina quando a fila está vazia, retornando os processos na ordem que foram concluídos.